

OpenShift Essential Training: Container Orchestration and Deployment

Note: How Kubernetes helps in orchestration and managing containerized applications

Kubernetes: Open-source platform for automating, deployment, scaling and management of containerized applications.

Key Kubernetes Components

- Nodes
- Pods
- Services
- Deployment
- Namespaces

Kubernetes Architecture

Master Node

API server

Controller manager

Scheduler

Etc

Worker Node

Kubelet

kube-proxy

Container runtime

Introduction to OpenShift

- Enterprise-grade container platform
- Built on Kubernetes with Red Hat features

- ❖ Security
- ❖ Monitoring
- ❖ Remote management
- ❖ Auditing

1. **Console** - Web interface for managing OpenShift
2. **Infranode** - Behind the scenes, Networking, Load balancing, storage
3. **Project** - Organize apps and services with controlled access

Routes: Link apps by mapping URLs to your services

Template: Blueprint to deploy application stack

Secret and ConfigMaps: Securely store data and configuration files

Setting Up the Lab Environment

Red Hat OpenShift Local : Faster way to start building OpenShift clusters

OpenShift Local Benefits

- Production-like environment for testing apps locally
- Minimal configuration
- Ultimate learning and prototyping

OpenShift Local

- Create microservices
- Build them into images
- Run in Kubernetes-hosted containers

Lab Environment Prerequisites

- 4 CPUs (vCPUs)
- Minimum 12 GB of Memory (RAM)
- Minimum 35 GB of Storage
- Red Hat account (<https://developers.redhat.com/products/openshift-local/overview>)
- Windows 10 Fall Creators Update (or later)
- macOS 10.12 Sierra (or newer)
- Red Hat Enterprise Linux 8 (or later)

Note: Go to Red Hat Developer website and download Red Hat OpenShift Local for windows, then install the setup then,

Open PowerShell with administrator right

#crc setup - To install and configure OpenShift locally

#crc start -

To Start (when you run this command then show you username and password and URL also to start your learning)

Managing Containers and Pods in OpenShift

Accessing OpenShift

#oc login <cluster>

#oc project <project_name>

#oc new project <project_name>

Deploying Containers

```
#oc new-app <app-name>
```

```
#oc apply-f deployment.yaml
```

Viewing and Deleting Pods

```
#oc get pods
```

```
#oc describe pod <pod_name>
```

```
#oc delete pod <pod_name>
```

```
#oc delete deployment <deployment_name>
```

Troubleshooting and Monitoring Pods

```
#oc logs <pod_name>
```

```
#oc logs <pod_name> -f
```

```
#oc debug <pod_name> -f
```

```
#oc top pods
```

```
#oc exec <pod_name>
```

LAB 1:

```
#crc start
```

```
#oc login -u developer https://api.crc.testing:6443
```

```
#oc new-project myproject - To create new project
```

```
#oc ne-app httpd
```

```
#oc status
```

```
#oc get all
```

```
#oc describe pod httpd-64u6979cc4-dzw5h
```

```
#oc logs httpd-64u6979cc4-dzw5h
```

```
#oc delete project myproject
```

Working with Container Images

Container Image: Application and ingredients

Note: images can exist without containers, but containers depend on images.

Image: Please where you can download and share container images.

Image Registries: Places where you can download and share container images.

- Quay.io
- Red hat Registry
- Docker Hub
- Skopeo

Skopeo:

```
#skopeo login
```

```
#skopeo inspect
```

```
#skopeo delete
```

```
#skopeo copy
```

OC Image:

```
#oc image info
```

```
#oc image append
```

```
#skopeo inspect docker://docker.io/library/httpd:latest
```

LAB 2:

```
#crc start
```

```
#oc login -u kubeadmin -p *****
```

```
#oc new-project myproject
```

```
#oc run myhttpd --image=docker.io/library/httpd:latest --restart=Never --port=80
```

```
#oc get pods
```

```
#oc describe pod myhttpd
```

```
#oc image info
```

Challenge prerequisites

- OpenShift local (CRC) setup
- Internet connectivity

Challenge Tasks

- Create a project called **myproject**
- Inspect the image using Skopeo at `docker.io/library/httpd:latest`
- Run a pod name **myhttpd** in OpenShift with the image
- View detailed information on the pod and its image
- Clean up all created resources

Deploying Your First Application in OpenShift 🤪 😊

#crc start - To start OpenShift in windows

#oc login -u developer

#oc new-project myproject

#skopeo inspect docker://docker.io/library/httpd:latest

#oc run myhttpd --image=docker.io/library/httpd:latest --restart=Never

#oc get pods

#oc describe pod myhttpd

#oc image info docker.io/library/httpd:latest

#oc image info docker.io/library/httpd:latest --filter-by-os linux/amd64

#oc delete project myproject

OpenShift Essentials

- Pods
- Deployments
- Deployment configs
- Replica sets
- Jobs

Networking Resources

- Services
- Routes
- Ingress

Storage Resources

1. Persistent volumes
2. Persistent volume claims
3. Storage classes

Configuration

- ConfigMaps
- Secrets

Scaling

- Horizontal autoscaler
- Cluster autoscaler

OpenShift Templates

- Collection of resources
- Use placeholders
- Written in YAML or json
- Customizable

oc process: Substitutes variables with values to create resource definition.

Oc apply: Ensures resources are created and deployed to your cluster

```
#crc start
```

```
#oc login -u kubeadmin -p *****
```

```
#oc new-project templates
```

```
#oc get templates -n openshift
```

```
#oc process --namespace=openshift httpd-example
```

```
#oc process --namespace=openshift httpd-example -p NAME=myhttpd -p  
NAMESPACE=templates | oc apply -f
```

```
#oc get all
```

```
#oc describe route myhttpd
```

Managing Application Lifespans

- Run 24/7
- Process tasks on demand
- Performs regular backups

Kubernetes/OpenShift Resources

- **Jobs:** Run a one-time task
- **Cronjobs:** Reminders to do something regularly
- **Deployment:** Keep things always running so there aren't interruptions

#crc start

```
#oc login -u developer
```

```
#oc new-project jobs
```

```
#oc create job myjob --image=busybox -- /bin/sh -c "echo hello from job"
```

```
#oc get jobs
```

```
#oc create cronjob mycronjob --image=busybox --schedule="*/1 * * * *" -- /bin/sh -  
c "Hello from cronjob"
```

```
#oc get cronjobs
```

```
#oc create deployment mydeployment --image=nginxinc/nginx-unprivileged
```

```
#oc gets deployments
```

```
#oc get all
```

```
#oc get pods
```

Introduction to OpenShift Networking

- OpenShift uses software defined networking (SDN) to create a flexible network.

OpenShift SDN

- Ensures communication between pods
- Automates traffic management
- Allows updates with no downtime

OpenShift Networking Components

- Pods
- Services
- Routes

OpenShift Service Types

- Cluster Ips
 - NodePort
 - LoadBalancer
 - ExternalName
- Pod has the application
 - Service exposed internally only
 - Route provides DNS for external

```
#crc start
```

```
#oc login -u developer
```

```
#oc new-project services
```

```
#oc create deployment myapp --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc expose deployment myapp --port=8080 --name=myapp-service
```

```
#oc get services
```

```
#oc run test-client --image=quay.io/fedora/fedora:latest --rm -it --restart=Never --bash
```

```
#curl http://myapp-service:8080 -
```

This command run inside the deployment running container.

Exposing Applications

- Making them accessible for other people in OpenShift

Ways to Expose

What For?

ClusterIP

Internal Only

NodePort

Limited external access

Route

Public access

Scaling in OpenShift

1. **Manual** - App has a set number of replicas
2. **Automatic** - Creates replicas based on specific criteria

```
#oc login -u developers
```

```
#oc new-project scaling
```

```
#oc create deployment myapp --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc scale deployment myapp --replicas 3
```

```
#oc get pods
```

```
#oc expose deployment myapp --port=8080 --name=myapp-service
```

```
#oc get services
```

```
#oc expose svc myapp-service
```

```
#oc get route
```

Secure Routes

- Encrypted traffic
- Protection from unauthorized access
- Ensures confidentiality and integrity

OpenShift	Situation
Edge termination	Entrance security check
Re-encrypt	Secure package delivery
Passthrough	VIP with security escort

Route type	Security level	Best for
Edge	Moderate	General apps
Re-encrypt	High	Sensitive data
Passthrough	Highest	Apps with strict security needs

- Always use HTTPS
- Choose the right route type
- Keep your apps secure

```
#crc start
```

```
#oc login -u developer
```

```
#oc new-project secure-routes
```

```
#oc create deployment myapp --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#mkdir certificates
```

```
#cd certificates
```

```
#openssl req -x509 -nodes -days 365 -newkey rsa:2048 -subj "/CN=myapp-ssl.apps-crc.testing" -keyout tls.key -out tls.crt
```

```
#oc expose deployment myapp --port=8080 --name=myapp-service
```

```
#oc get services
```

```
#oc create route edge myapp-route --service=myapp-service --cert=tls.crt --key=tls.key --ca-cert=tls.crt
```

```
#oc get routes
```

Network Policies:

- Use labels to define rules
- Apply rules to incoming (ingress) and outgoing (egress) traffic
- By default, they deny all traffic

Without Network Policies:

- All pods can talk to each other
- No security

With Network Policies:

- Manage which pods can communicate with others
- Improved security and isolation

Sample Network Policy Example

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-app-a
spec:
  podSelector:
    matchLabels:
      app: app-b
  policyTypes:
  - Ingress
  Ingress:
  - from:
    - podSelector:
        matchLabels:
          app: app-a
```

```
#oc login -u developer
```

```
#oc new-project policies
```

```
#oc create deployment app-a --image=nginxinc/nginx-unprivileged
```

```
#oc create deployment app-b --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc expose deployment app-a --port=8080 --name=svc-a
```

```
#oc expose deployment app-b --port=8080 --name=svc-b
```

```
#mkdir policies
```

```
#cd policies
```

Note: Please open the Link to use networking.yaml file

<https://github.com/brahamdev1995/DevOps-Project-2026/blob/main/networking.yaml>

```
#oc apply -f networking.yaml
```

```
#oc run test-client --image=quay.io/fedora/fedora:latest --rm -it --restart=Never --bash
```

(This command is used to go inside the running container)

Fh74y#curl <http://svc-b:8080> - This command fire inside the running container.

Fh74y#curl <http://svc-a:8080> - Now you see the error

Challenge 🤔

Deploy a Containerized Web Server in OpenShift

Challenge Prerequisites:

- Lab environment
 - ✓ OpenShift local (CRC) setup
- Internet connectivity

Challenge Tasks

1. Deploy NGINX web server: nginxinc/nginx-unprivileged
2. Scale application with three replicas and expose it.
3. Secure application with TLS using secure edge route.
4. Block incoming traffic to web server from other pods.
5. Clean up workspace.

Solution 😊

```
#oc login -u developer
```

```
#oc new-project challenge
```

```
#oc create deployment nginx --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc scale deployment nginx --replicas=3
```

```
#oc get pods
```

```
#oc expose deployment nginx --port=8080 --name=nginx-service
```

```
#oc get services
```

```
#openssl req -x509 -nodes --days 365 -newkey rsa:2048 -subj "/CN=nginx.apps-crc.testing" -keyout tls.key -out tls.crt
```

```
#ls
```

```
#oc create route edge nginx-route --service=nginx-service --cert=tls.crt --key=tls.key --ca-cert=tls.crt
```

```
#oc get route
```

<https://github.com/brahamdev1995/DevOps-Project-2026/commit/e95981598a261acfa1164b2051b6ddd8fc13cda0>

Note: Please find the attachment using above link this is policy.yml file

```
#oc apply -f policy.yml
```

```
#oc run test-client --image=quay.io/fedora/fedora:latest --rm -it --restart=Never --bash
```

```
bash-5.2$ - Now you are inside the running container
```

```
bash-5.2$ curl http://nginx-service:8080
```

```
bash-5.2$ exit - To exit from running container
```

```
#oc delete project challenge - To delete project challenge
```

Understanding OpenShift Storage:

- Containers have **ephemeral** (temporary) storage.
- They don't have persistent storage.

Ephemeral Storage

- Tied to pod's lifecycle
- Doesn't store logs, databases, or application data

Persistent Storage

- Data remains even if pod is deleted
- Supports databases, logs, and application state

Persistent Volume (PV)

- A physical storage space in the cluster, like a hard drive or cloud storage
- PV like Reserved room

Persistent Volume Claim (PVC)

- PVC like Requested room
- A request for storage from a pod
- PVC requests storage and access.
- OpenShift binds it to a matching PV.

Reclaim Policy

- What happens when a persistent volume or PV is no longer needed.

Reclaim Policy Types

- | | | |
|-----------|---|--|
| 1. Delete | - | Storage automatically deleted when PVC removed |
| 2. Retain | - | Storage remains after PVC deleted |

Storage Classes:

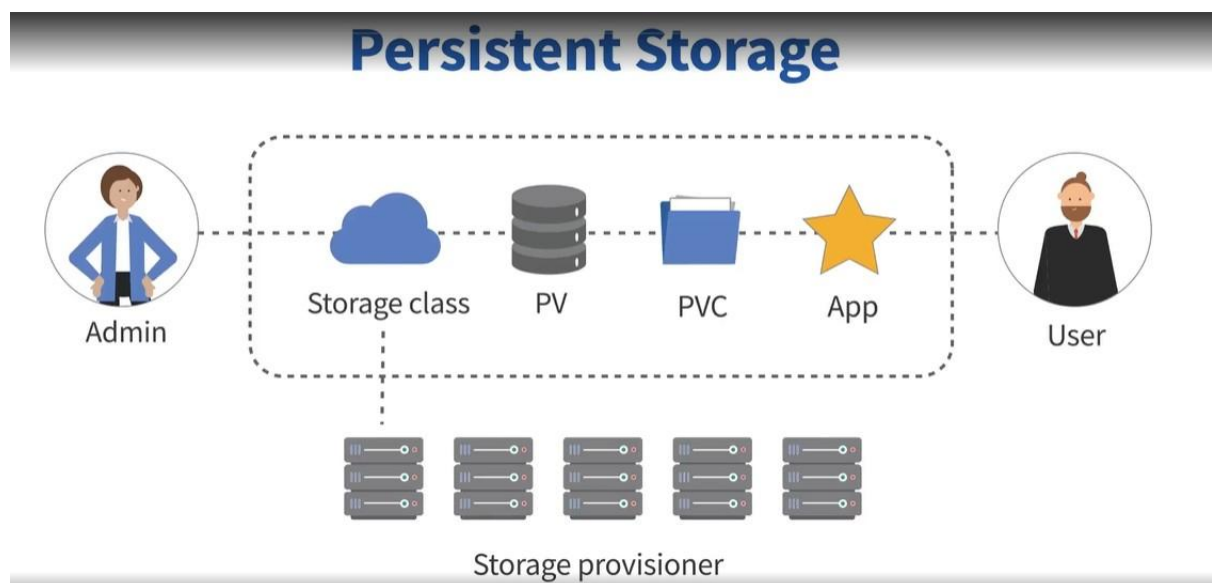
Why Use Storage Classes?

Storage Classes

- Automatic storage setup
- Different options based on workload
- Simplifies storage management

Static Storage

- Manually create PVs
- Hard to scale dynamically
- Storage requests may be delay



```
#oc new-project storage
```

<https://github.com/brahamdev1995/RHCSA-RHCE-DevOps-Notes-2026/blob/main/mypvc.yml>

(Please find the mypvc.yml file from my github account then go ahead.)

```
#oc apply -f mypvc.yml
```

```
#oc get pvc
```

```
#oc create deployment myapp --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc set volume deployment/myapp --add --name=myvolume --mount-path=/data --claim-name=mypvc
```

```
#oc get pvc
```

```
#oc get pods
```

```
#oc describe pod myapp
```

OpenShift Secrets:

- A solution to manage sensitive information without exposing it
- Secrets are Kubernetes objects designed to store sensitive data.

A World without Secrets

- Hard-coded credentials
- Passwords in version control
- Sensitive data stored in plain text
- Data leaked through logs

Why Secrets?

- Encrypted
- Stored securely in OpenShift
- Access controlled via RBAC
- Mounted as files

Lab:

```
#oc login -u developer
```

```
#oc new-project secrets
```

```
#oc create secret generic mysecret --from-literal=username=my-user --from-literal=password=secret
```

```
#oc get secrets
```

```
#oc create deployment secret-app --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc set volume deployment/secret-app --add --name=secret-vol1 --type=secret --secret-name=mysecret --mount-path=/usr/share/nginx/html/secret --read-only=true
```

```
#oc rollout status deployment/secret-all
```

```
#oc get pods
```

```
#oc exec -it secret-app-8eye9e00e03 -- cat /usr/share/nginx/html/secret/username  
#oc delete project secrets
```

Limiting Resource Use:

Why Resource Management?

- Sustainable allocation of resources
- Prevents resource overuse
- Maintains cluster performance and stability

1. **Resource Quotas:** Restrict resources a namespace can consume
2. **LimitRanges:** Prevents pods from requesting too little or too much
3. **Limits:** Ensure specific workloads don't consume too much

```
#oc login -u developer
```

```
#oc new-project limits
```

<https://github.com/brahamdev1995/RHCSA-RHCE-DevOps-Notes-2026/blob/main/quota.yml>

Note: Please find the quota.yml file from my GitHub link below above.

```
#oc apply -f quota.yml
```

<https://github.com/brahamdev1995/RHCSA-RHCE-DevOps-Notes-2026/blob/main/limits.yml>

Note: Please download the limits.yml file from my GitHub Repo.

```
#oc apply -c limits.yml
```

```
#oc create deployment test --image=nginxinc/nginx-unprivileged --replicas=6
```

```
#oc get pods
```

```
#oc get pods
```

 - Now you can see only two pod are created and run because resources quota is applied.

OpenShift role-based access controls (RBAC) control who can go where and do what.

Role-Based access controls (RBAC) User Types

1. Regular users
2. System users
3. Service accounts

Authorization Process in OpenShift

Rules 1: Actions allowed for objects or object groups

Roles 2: Sets of rules

Bindings 3: Assigning users or groups to a role

Role Level	Description
Cluster role	Can manage the entire cluster
Local role	Can only manage elements at project level

Default Roles

1. Admin
2. Basic user
3. Cluster admin

```
#crc start
```

```
#oc login -u kubeadmin -p *****
```

```
#oc new-project roles
```

```
#oc get rolebindings
```

```
#oc login -u developer
```

Note: One thing to note here is that when we login with the developer user, it appears that you do not have any project, but after logging in with the kubeadmin user, it gives us the notification that you have many projects.

```
#oc adm policy add-role-to-user admin developer -n roles
```

```
#oc adm policy add-cluster-role-to-user-admin developer
```

```
#oc login -u developer
```

```
#oc project roles
```

```
#oc get rolebinding
```

```
#oc create deployment myapp --image=nginxinx/nginx-unprivileged
```

```
#oc get pods
```

Challenge:

Deploy a Containerized Database

Challenge Prerequisites

- OpenShift local (CRC) setup
 - Internet connectivity
-
- New payment processing system
 - Requires widely available database for secure customer transactions

Challenge Tasks

1. Create a 1 GB PersistentVolumeClaim (PVC) called db-pvc.
2. Create a secret called db-credentials with the following entries:

```
POSTGRESQL_USERNAME=myuser
POSTGRESQL_PASSWORD=mypass
POSTGRESQL_DATABASE=mydb
```

3. Deploy the database “postgrs” from bitnami/postgresql:15
4. Set limits to prevent the database from using too many resources by allowing it to request 200CPU and 512 MB memory while capping it at 500 CPU and 1GB memory.

Solution:

1. Create a 1 GB PersistentVolumeClaim (PVC) called db-pvc.

```
#oc login -u developer
```

```
#oc new-project challenge
```

```
#mkdir challenge
```

```
#cd challenge
```

```
#vim mypvc.yml
```

<https://github.com/brahamdev1995/RHCSA-RHCE-DevOps-Notes-2026/blob/main/mypvc1.yml>

Please Download the mypvc.yml file from my git repository for practical purposes.

```
#oc apply -f mypvc.yml
```

```
#oc get pvc
```

2. Create a secret called db-credentials with the following entries:

POSTGRESQL_USERNAME=myuser
POSTGRESQL_PASSWORD=mypass
POSTGRESQL_DATABASE=mydb

```
#oc create secret generic db-credentials --from-  
literal=POSTGRESQL_USERNAME=myuser --from-  
literal=POSTGRESQL_PASSWORD=mypass --from-  
literal=POSTGRESQL_DATABASE=mydb
```

3. Deploy the database “postgrs” from bitnami/postgresql:15

```
#oc create deployment postgress --image=bitnami/postgresql:15
```

```
#oc get pods
```

```
#oc set env deployment/postgress --from=secret/db-credentials
```

```
#oc get pods
```

4. Set limits to prevent the database from using too many resources by allowing it to request 200CPU and 512 MB memory while capping it at 500 CPU and 1GB memory.

```
#oc set resources deployment/postgresss --limits=cpu=500m,memory=1Gi --  
requests=cpu=200m,memory=512Mi
```

```
#oc set volume deployment/postgress --add --name=db-storage --mount-  
path=/bitnami/postgresql --type=persistentVolumeClaim --claim-name=db-pvc
```

```
#oc get pods
```

```
#oc get pvc
```

Probes in OpenShift:

A health monitoring system for your app

Health Probes

- Let the cluster check app status by regular probing
- Crash mitigation
- Failure
- Monitors for pod failures
- Determine when a new replica is ready for requests

Probe Types

- Liveness - Checking if container is ready to serve requests
- Readiness - Check if container is still running
- Startup - Check if container has started successfully

Test Type

Description

HTTP GET	Sends HTTP request to path and port
TCP socket	Tries to open TCP connections to specific port
Exec command	Performs simple task

```
#oc login -u developer
```

```
#oc new-project probes
```

```
#oc create deployment myapp --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc set probe deployment/myapp --readiness --get-url=http://:8080/ --initial-delay-seconds=5
```

```
#oc set probe deployment/app --liveness --get-url=http://:8080/ --initial-delay-seconds=5
```

```
#oc describe deployment myapp
```

```
#oc set probe deployment/mapp --liveness --get-url=http://:8080/invalid-path --initial-delay-seconds=5 --failure-threshold=3
```

```
#oc get pods
```

```
#oc get pods -w
```

Application Autoscaling:

The horizontal pod autoscaler changes the number of pods based on CPU or other metrics.

Benefits of Autoscaling

- Keeps app responsive under load
- Saves resources when demand is low
- Works continuously in the background

Condition 1: Usage > Threshold

Condition 2: Scale up | Scale down

```
#oc login -u developer
```

```
#oc new-project hpa
```

```
#oc create deployment myapp --image=nginxinc/nginx-unprivileged
```

```
#oc get pods
```

```
#oc set resources deployment myapp --requests=cpu=100m --limits=cpu=200m
```

```
#oc autoscale deployment myapp --cpu-percent=50 --min=1 --max=5
```

```
#oc get hpa
```

Cluster Administration:

Use the **web console** for cluster administration tasks.

Planes take off > Apps deployed

No runway traffic > Resources balanced, alerts, updates visible

Tower view > Full control of what's happening

Visibility in OpenShift Web Console

- Cluster status metrics and alerts
- How much memory is being used
- If a node isn't responding

Resources Management

- Monitor node performance
- See what pods are running and where

App Management

- Create and manage projects
- Deploy app from image or GIF
- Access to cluster setting and updates
- Install and monitor operators
- View logs, metrics, and events

Why Use OpenShift Web Console?

- Visual, intuitive interface
- Ideal for daily cluster management

#crc start

Note: Then copy the URL and paste the any browser and use GUI mode of OpenShift

CHALLENGE

- Deploy an application using the Web Console

Challenge Prerequisites

- OpenShift local (CRC) setup
- Internet connectivity

Deploy test version of product website with OpenShift web console

Team needs: basic web server and NGINX

Challenge Tasks

1. Access OpenShift web console.
2. Create a new project named **product-demo**
3. Deploy NGINX application named **demo-web**
4. Scale the application to three replicas.
5. Expose the application.

This theory and practical written by

(Braham Dev) Linux Administrator.

